

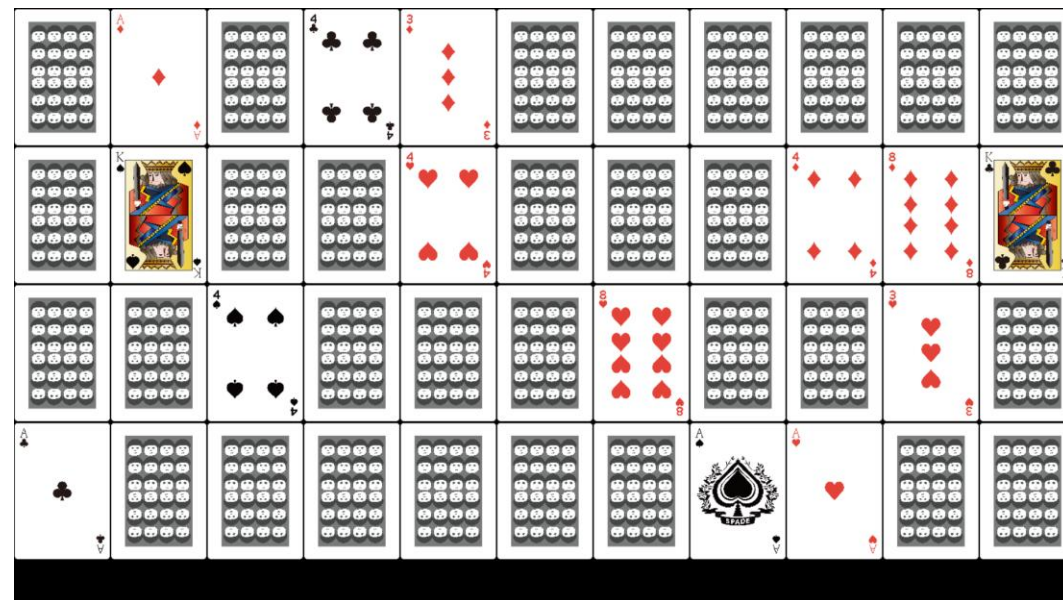
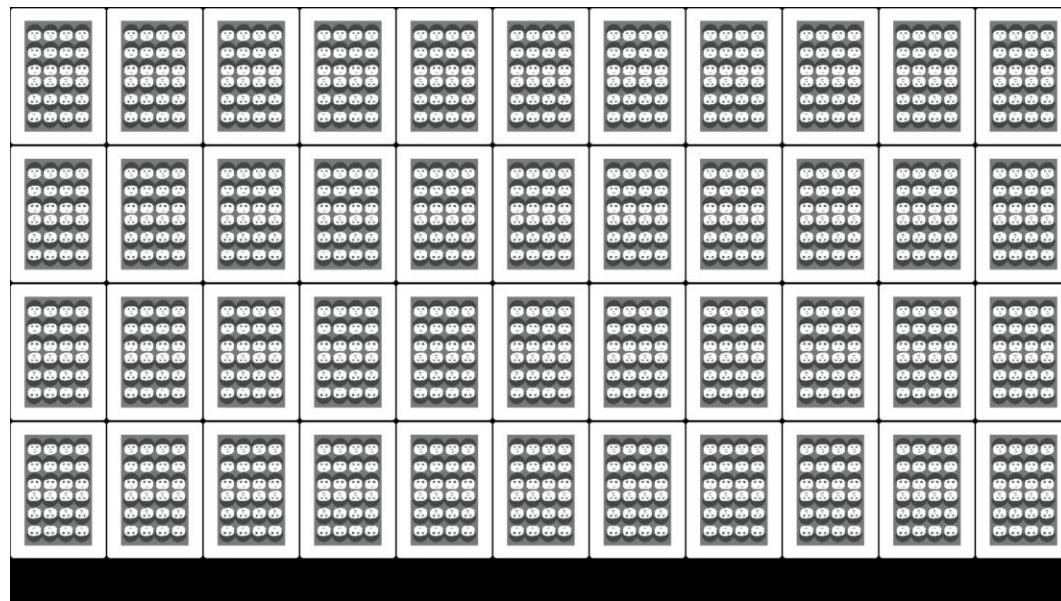
タイトル : 神経衰弱
ジャンル : トランプゲーム
制作期間 : 3年次の4月から5月
制作人数 : 1人
担当箇所 : プログラムすべて
使用言語 : C++
使用ツール : VisualStudio2026, DxLib
対応プラットフォーム : Windows

ゲーム内容

同じ数と色のカードを2枚選択して、すべてのカードを表向きにするゲームです。

制作意図

CSVファイルの読み込みの学習を目的に作りました。



アピールポイント①

該当スクリプト：ソースコード/Manager/CardManager

拡張性の高い画像読み込みの実現のためCSVファイル読み込みを行った
Excelで作成したCSVファイルから、トランプの数字や画像ファイルを読み込むこと
で修正や拡張がしやすいようにした。

CSVファイルにはIDと数字、色、
画像ファイルの名前が書いてあります。

ID	Number	Color	FilePath
0	1	black	torannpu-illust1.png
1	2	black	torannpu-illust2.png
2	3	black	torannpu-illust3.png
3	4	black	torannpu-illust4.png
4	5	black	torannpu-illust5.png
5	6	black	torannpu-illust6.png
6	7	black	torannpu-illust7.png

読み込んだ1行から改行を取り除く
ことで、行の加工をやすくしました。
読み込んだ1行をカンマ区切りで1つ
ずつ読み込むことに苦劳しました。

```
std::stringstream ss(readBuf); // 読み込んだ1行を一時保存する配列
std::string line; // カンマで区切った文字列を一時保存する
std::string columns[4]; // 各要素を入れる

// 各要素の代入
int i = 0;
while (std::getline(ss, line, ',') && i < 4)
{
    columns[i] = line;
    i++;
}
```

アピールポイント②

該当スクリプト：ソースコード/Manager/CardManager

カードのデータの複雑化を防止するためにデータの分割を行った

カードには大きく分けて「ロジック用データ」と「描画用のデータ」の2種類のデータがあり、それらをまとめてしまうと管理しづらいため分割しました。

生成IDやフラグなどロジックに必要なカード1枚ずつのデータはすべてここに入ります。

```
struct CardData
{
    int id;           // 生成ID
    int number;       // 1~13の数字を表す
    bool isRed;       // 赤色かどうか (true: 赤色, false: 黒色)
    bool isBack;      // 裏向きかどうか (true: 裏, false: 表)

    // アニメーションの情報
    float progress = 0; // 進行度 (0.0~1.0)
    bool isFlipping = false; // アニメーション中か (true: true, false: false)
    bool isSwitched = false; // 裏表が切り替わったか (true: true, false: false)
};
```

画像ハンドルなど描画に必要なデータはここにまとめています。

```
// カードの描画用のデータ
struct CardRenderData
{
    int handle[52]; // 52枚の画像ハンドル
    int backCardHandle; // 裏向きカードの画像ハンドル
    int imageWidth; // 元画像の幅
    int imageHeight; // 元画像の高さ
    double baseScaleX; // 幅の拡大率
    double baseScaleY; // 高さの拡大率
    const int row = 4; // 行数
    const int col = 13; // 列数
    const int width = 140; // カードの幅 (目標値)
    const int height = 200; // カードの高さ (目標値)
};
```

課題と解決方法

課題：選んだカードを覚える時間がない

カードを2枚選択したあとに、カードが異なる場合はすぐにカードが裏向きに戻されるため、2枚目に選択したカードの色と数字を確認できない問題がありました。

原因は、2枚選択したあとに1フレームでカードがそろっているかチェックし、結果を出しているためでした。

解決方法：待機時間をつくる

2枚目を選択したあとに、何フレームか待ったあとにカードのチェックに入るようにしました。

結果：覚える時間になった

しかし、待機時間に何も画面が動かないのはユーザーに対して不安感を持たせるため、フリップアニメーションを入れることでプログラムが動いていることを示すようにしました。

振り返り

学び：ゲームの状態の管理と制御の重要性

理由：

カードの選択とチェックと終了判定の流れが見づらくなり、enumとswitch文を使用して状態管理をしたところ管理しやすくなるだけでなく各状態ごとに処理を分けることができるようになったから。

今後に向けて：

ゲームやキャラクターなど状態を複数持ち、その流れに一定の規則性がある場合には状態遷移の処理を持たせることで処理を見やすくすることでチーム制作で開発しやすくなると感じました。

そのためにステートマシンについて学習したいと思いました。